

楽譜データ・発音情報仕様書

WBS 116 / 128 / 129 / 130 対応版

25G1053 齋藤翔太

2026 年 4 月 29 日

本書の位置づけ

本書は、チーム 23「Arduino オーケストラ」における齋藤担当範囲（楽譜データ形式、4 パート振り分け、音高・拍の表現、Arduino から PC へ送る発音情報）の仕様書である。第 2 回 WBS のうち、表 1 に示す 4 タスクを対象とする。

表 1 本書が担当する WBS

WBS	内容
116	楽譜データ形式 基本設計。4 種類の楽器パートを Arduino のメモリに埋め込む方針を決める。
128	楽譜データ詳細設計。課題曲を 4 種類の楽器パートに振り分ける。
129	楽譜データ詳細設計。テンポ、音の高さ、拍をどんな形でデータに書くか決める。
130	楽譜データ詳細設計。Arduino から PC へ送る音データの中身と送信タイミングを決める。

既存資料との整合を優先し、通信と楽器ノード処理は塩澤担当の `arduino_塩澤.pdf` に合わせる。したがって、本書ではカンマ区切りテキストではなく、**共通ヘッダ 12 バイト + NOTE ペイロード 8 バイト = 20 バイト固定長バイナリ**を採用する。楽譜データも `ScoreEvent` 形式へ統一する。

1 ルーブリック対応方針

システム設計書ルーブリックで 10 点を狙うため、本書では表 2 の内容を明示的に記述する。齋藤担当範囲だけで閉じるのではなく、結合相手である Arduino ノード、Processing、PC、スピーカ

との関係まで示す。

表 2: ルーブリック項目と本書の対応

評価項目	本書で満たす内容
基本設計：機能一覧とシステム構成図	FBS の F3「楽譜進行」と F4「発音情報配信」に対応する機能を列挙し、ユーザ目線の構成図とノード間のデータ流れを示す。
基本設計：操作インターフェースと状態遷移	ユーザ操作、LED 表示、楽器ノード状態遷移、テンポ変更時の挙動を示す。
詳細設計：ハードウェア設計	楽譜データ仕様が前提とする Arduino, USB Serial, PC, スピーカ, WiFi 接続, 追加ストレージ不要の条件を示す。
詳細設計：ソフトウェア設計	ScoreEvent, ProjectConfig, NotePacket, 関数 API, ファイル構成, 実装例を示す。
詳細設計：処理フローの設計	起動, 同期, 輪唱入り待ち, BEAT 受信, 休符, NoteOn/NoteOff, テンポ変更, パケロス時自走, 曲終了までの処理を条件分岐付きで示す。

2 基本設計

2.1 機能一覧と FBS 対応

本担当範囲は、チーム全体 FBS のうち F3「楽譜進行」と F4「発音情報配信」に対応する。機能一覧を表 3 に示す。

表 3: 齋藤担当範囲の機能一覧

ID	機能	役割
F3.2	楽譜データ保持	各楽器ノードが自分のパートの楽譜を C 配列として保持する。SD カード等の外部ストレージは使わない。
F3.3	BEAT 同期での音符進行	指揮者ノードから届く BEAT 番号とマスタ時刻に合わせ、現在発火すべき楽譜イベントを選ぶ。
F3.4	NOTE イベント生成	音符開始時に NoteOn, 発音長経過時に NoteOff を生成する。休符は待機だけ行い NOTE を送らない。

ID	機能	役割
F3.5	パート固有ロジック	partId, startBeatNo, 楽譜配列をノードごとに差し替え, 4 声の輪唱とリズムを成立させる.
F4.1	NOTE パケット生成	発音情報を partId, noteNumber, velocity, gate, durationMs に変換する.
F4.2	Processing への送信	20 バイト固定長バイナリを USB Serial で自パートの PC へ送信する.

2.2 ユーザ目線のシステム構成図

ユーザは指揮者ノードを手に持ち、振る速さを変える。楽器ノードは楽譜データに従って発音指令を PC へ送り、PC 側の Processing が音色を合成してスピーカから出力する。

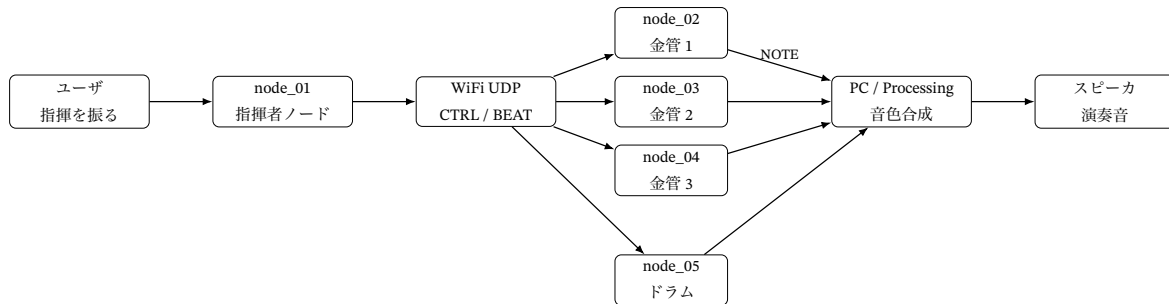


図 1 ユーザ目線の構成図

2.3 課題曲とパート振り分け

本仕様では、課題曲候補として既存たたき台と同じ「かえるのうた」を採用する。短い輪唱曲であり、指揮テンポ変更の確認がしやすい。第3回ミーティングで別曲へ変更する場合も、本書の ScoreEvent 形式とパート割当規則はそのまま使う。

表 4 4パート振り分け

ノード	partId	役割	入り拍	楽譜内容
node_02	0x02	金管 1	0 拍	主旋律を曲頭から演奏する。
node_03	0x03	金管 2	4 拍	主旋律を 4 拍遅れて演奏する。
node_04	0x04	金管 3	8 拍	主旋律を 8 拍遅れて演奏する。
node_05	0x05	ドラム	0 拍	1 拍ごとのリズム伴奏を演奏する。

金管 3 声とドラム 1 声に分けることで、ADR-0004 の「主旋律 3 + リズム 1 以上」という制約と、塩澤仕様書のノード構成を同時に満たす。

2.4 操作インターフェースと状態遷移

楽譜データ担当範囲に物理スイッチは追加しない。ユーザ操作は指揮者ノード側に集約し、楽器ノードは LED 表示で状態を返す。

表 5 ユーザ操作と楽器ノード側の挙動

ユーザ操作	入力される情報	楽器ノードの挙動
電源を入れる	USB 給電, WiFi 接続開始	Idle から WaitSync へ進み, CTRL 受信を待つ。
指揮を振り始める	BEAT と BPM が届く	startBeatNo に達したパートから演奏を開始する。
速く振る／遅く振る	CTRL の BPM 値が変化する	次の音符長計算から新しい BPM を使う。楽譜データ自体は変更しない。
一時的に BEAT が欠ける	BEAT 未受信時間がしきい値を超える	直前 BPM で SelfRun し、復帰後は実 BEAT に戻る。
曲が終わる	最終イベントまで発火	デモ用途では曲頭へ戻りループ演奏する。提出時に停止へ変更可能。

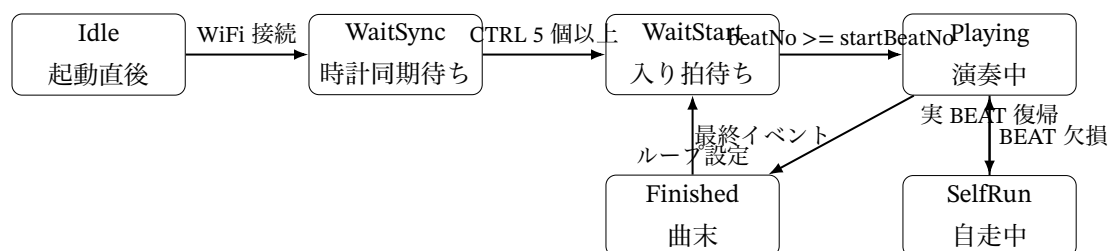


図 2 楽器ノードの状態遷移

3 詳細設計：ハードウェア条件

3.1 使用する機器

齋藤担当範囲は主にソフトウェア仕様であるが、実装に必要なハードウェア条件を表 6 に定める。

表 6: 楽譜データ仕様が前提とする機器

機器	型番・構成	本仕様での役割
楽器ノード	Arduino UNO R4 WiFi × 4	node_02～node_05. 楽譜配列を保持し, NOTE を送信する.
指揮者ノード	Arduino 系マイコン × 1	CTRL / BEAT を WiFi UDP で配信する. 楽譜 データは持たない.
PC	Mac または PC × 1～4	Processing を実行し, USB Serial から NOTE を 受け取る.
接続ケーブル	USB Type-C 等	楽器ノードと PC を 1 対 1, または USB ハブ経 由で接続する.
音声出力	PC 内蔵または外部ス ピーカ	Processing が合成した音を出力する.
ストレージ	追加なし	楽譜は Arduino のプログラムメモリに埋め込 むため, SD カードは使わない.

3.2 接続関係

CTRL / BEAT は WiFi UDP, NOTE は USB Serial で分ける. 楽譜データは Arduino 内部に固定配
置し, 演奏中に PC から読み込まない.



図 3 ハードウェア接続関係

配線上の注意点は以下である.

- 楽器ノードは, PC から USB 給電される想定とする.
- NOTE は USB Serial に流すため, WiFi 側の混雑が Processing への発音指令に直接影響しにくい.
- PC が 1 台構成の場合は, 4 つの USB Serial ポートを開く. PC が 4 台構成の場合は, 各 PC が 1 ポートだけ開く.
- 楽譜データ担当として追加センサや追加配線は要求しない.

4 詳細設計：ソフトウェア仕様

4.1 ファイル構成

楽器ノードは共通コードを使い、ProjectConfig.h と score_data.cpp だけをパートごとに差し替える。

```
firmware/  
|-- node_02/  
|   |-- include/  
|   |   |-- ProjectConfig.h    // partId, startBeatNo, 楽器種別  
|   |   '-- score_data.h      // ScoreEvent の宣言  
|   '-- src/  
|       |-- applyPattern.cpp  // 楽譜進行  
|       '-- score_data.cpp    // node_02 用楽譜本体  
|-- node_03/                  // node_02 と同構造  
|-- node_04/  
'-- node_05/
```

4.2 楽譜イベント形式

楽譜は ScoreEvent 配列で表現する。小数の float 拍ではなく、固定小数 durationQ8 を使うことで、Arduino 側で扱いやすくする。

表 7 ScoreEvent のフィールド

フィールド	型	意味
beatAt	uint16_t	このイベントを発火する拍番号。各パートの入り拍からの相対値。
noteNumber	uint8_t	MIDI ノート番号。60=C4。0 は休符。
velocity	uint8_t	楽譜上の強さ。0~127。CTRL の velocity と合成する。
durationQ8	uint16_t	音の長さ。256 が 1 拍、128 が 0.5 拍、512 が 2 拍。
flags	uint8_t	bit0=NoteOn, bit1=NoteOff 拡張, bit2=休符。

```
struct ScoreEvent {  
    uint16_t beatAt;    // 発火する拍番号（パート開始からの相対拍）
```

```

uint8_t noteNumber;    // MIDI ノート番号. 0 なら休符
uint8_t velocity;      // 楽譜上の強さ 0~127
uint16_t durationQ8;   // 256 = 1 拍
uint8_t flags;         // bit0=NoteOn, bit2=休符
};

```

4.3 定数と表現ルール

表 8 音高・拍・休符の表現ルール

項目	ルール
音高	MIDI ノート番号で書く. C4 は 60, D4 は 62, E4 は 64, F4 は 65, G4 は 67.
休符	noteNumber = 0 かつ flags の休符 bit を立てる. NOTE は送信せず, 拍だけ進める.
拍	durationQ8 で書く. 四分音符=256, 八分音符=128, 二分音符=512.
テンポ	楽譜には BPM を埋め込まない. CTRL で届く BPM により, 実時間へ変換する.
輪唱の入り	ProjectConfig.h の startBeatNo で指定する. 楽譜配列自体は 0 拍始まりで再利用する.
NoteOff	durationQ8 から計算した時刻で applyPattern() が自動生成する.

4.4 パート別設定

```

struct ProjectConfig {
    uint8_t partId;          // 0x02~0x05
    uint16_t startBeatNo;    // 輪唱の入り拍
    uint8_t instrumentKind;  // 1=金管, 2=ドラム
    bool loopEnabled;        // デモ時は true
};

```

4.5 楽譜データ例

主旋律 A の冒頭は表 10 のように表す.

```

const ScoreEvent kScore[] PROGMEM = {

```

表 9 ProjectConfig のノード別値

ノード	partId	startBeatNo	instrumentKind	楽譜
node_02	0x02	0	金管	主旋律 A
node_03	0x03	4	金管	主旋律 A
node_04	0x04	8	金管	主旋律 A
node_05	0x05	0	ドラム	リズム伴奏

表 10 主旋律 A 「かえるのうた」冒頭

順番	beatAt	音名	noteNumber	durationQ8	意味
1	0	ド	60	256	1 拍鳴らす
2	1	レ	62	256	1 拍鳴らす
3	2	ミ	64	256	1 拍鳴らす
4	3	ファ	65	256	1 拍鳴らす
5	4	ミ	64	256	1 拍鳴らす
6	5	レ	62	256	1 拍鳴らす
7	6	ド	60	256	1 拍鳴らす
8	7	休符	0	256	1 拍待つ

```

{0, 60, 96, 256, 0x01}, // ド
{1, 62, 96, 256, 0x01}, // レ
{2, 64, 96, 256, 0x01}, // ミ
{3, 65, 96, 256, 0x01}, // ファ
{4, 64, 96, 256, 0x01}, // ミ
{5, 62, 96, 256, 0x01}, // レ
{6, 60, 96, 256, 0x01}, // ド
{7, 0, 0, 256, 0x04} // 休符
};

const uint16_t kScoreLength = sizeof(kScore) / sizeof(kScore[0]);

node_03 と node_04 は同じ主旋律配列を使い、startBeatNo だけで入りを遅らせる。これにより、楽譜配列の重複を減らし、修正時の不一致を防ぐ。

```

4.6 NOTE パケット仕様

Processing へ送る NOTE は、塩澤担当の通信仕様に合わせて 20 バイト固定長とする。共通ヘッダは全パケット共通で、NOTE ペイロードだけを表 11 に示す。

送信単位は次の 2 種類である。

表 11 NOTE ペイロード

offset	field	type	意味
12	partId	uint8_t	0x02～0x05. 発音元パート.
13	noteNumber	uint8_t	MIDI ノート番号. 休符は送らない.
14	velocity	uint8_t	0～127. 楽譜値と CTRL 値の合成後.
15	gate	uint8_t	1=NoteOn, 0=NoteOff.
16	durationMs	uint16_t	発音予定長さ. Processing 側の参考値.
18	予約	uint8_t[2]	0 で埋める.

- **NoteOn:** 音符イベント発火時に送る. gate=1.
- **NoteOff:** durationQ8 ぶん経過した時点で送る. gate=0.

休符イベントは NoteOn も NoteOff も送らない. ただし楽譜ポインタと拍の進行は行う.

4.7 内部 API

表 12: 楽譜進行で使用する内部 API

関数	役割
loadScoreEvent(index)	PROGMEM 上の kScore[index] を RAM 上の一時変数へ読む.
isRest(event)	noteNumber == 0 または休符フラグから休符か判定する.
durationQ8ToMs(durationQ8, bpm)	durationQ8 を現在 BPM に基づきミリ秒へ変換する.
mergeVelocity(score, ctrl)	楽譜上の velocity と指揮者側 CTRL の velocity を合成する. ストレッチ未実装時は CTRL=64 固定.
emitNoteOn(event)	pendingOn を立て, NOTE 送信モジュールへ渡すデータを作る.
emitNoteOff()	発音中の音を止めるため, pendingOff を立てる.
advanceScore(beatNo)	BEAT 番号から発火対象イベントを順に処理する.
buildNotePacket()	共通ヘッダと NOTE ペイロードを 20 バイト列にシリアルライズする.

5 詳細設計：処理フロー

5.1 全体フロー

楽器ノードは，入力，ロジック，出力の3フェーズで処理する．楽譜データに関するロジックは `advanceScore()` に集約する．

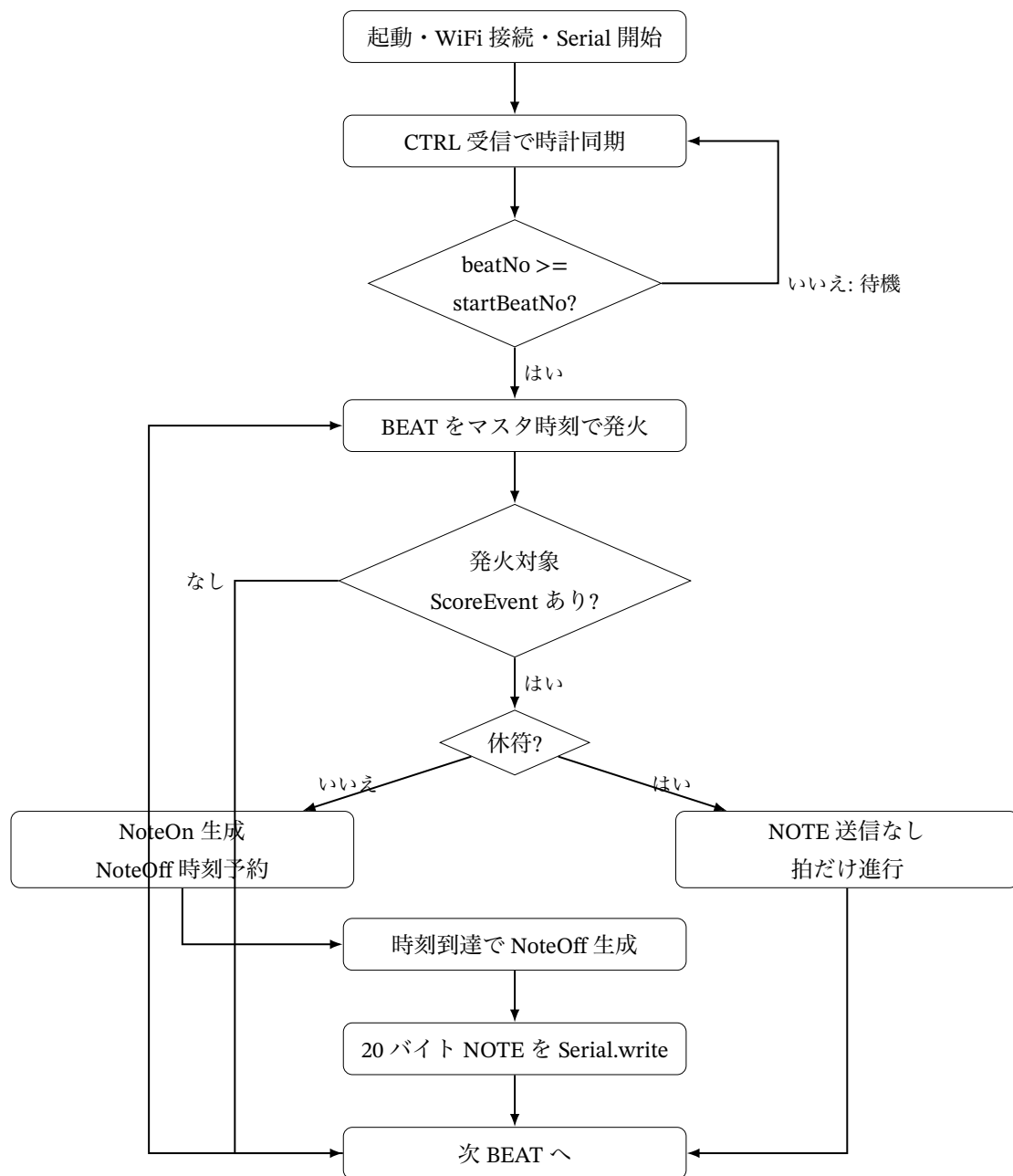


図 4 楽譜進行と NOTE 送信の処理フロー

5.2 BEAT 受信時の処理

1. 受信した BEAT の beatNo と playAtMasterMs をキューに入れる.
2. beatNo < startBeatNo の場合は、自パートの入り前なので演奏しない.
3. effectiveBeatNo = beatNo - startBeatNo を計算する.
4. kScore[currentEventIndex].beatAt <= effectiveBeatNo のイベントを順に読む.

5. 休符なら NOTE を送らず，currentEventIndex だけ進める。
6. 音符なら durationQ8ToMs() で発音長を求め，NoteOn を送る。
7. 発音中の音について，予約した noteOffAtMs を過ぎたら NoteOff を送る。

5.3 テンポ変更時の処理

楽譜データは拍単位で記録するため，テンポ変更時に配列を書き換える必要はない。CTRL で新しい BPM を受け取った後，次に発火する音符から以下の式で実時間を計算する。

$$durationMs = \frac{60000}{BPM} \times \frac{durationQ8}{256}$$

すでに鳴っている音については，発音開始時に予約した noteOffAtMs を優先する。これにより，途中で BPM が変わっても鳴りかけの音が不自然に切れない。

5.4 例外処理

表 13: 例外・分岐時の処理

状況	判定条件	処理
入り拍前	beatNo < startBeatNo	演奏せず待機する。
休符	noteNumber == 0	NOTE を送らず，次イベントへ進む。
BEAT 欠損	beatTimeoutMs 超過	直前 BPM から仮想 BEAT を作り SelfRun する。
遅延 BEAT	masterNow - playAtMasterMs が許容値超過	発音をスキップし，ログに警告を出す。
同じ BEAT の重複	seq または beatNo が処理済み	二重発音を防ぐため捨てる。
曲末	currentEventIndex >= kScoreLength	デモでは曲頭へ戻る。提出仕様では停止へ変更可能。
Serial 送信失敗	Serial.write() の戻り値が 20 未満	送信失敗ログを出し，次 NOTE で復帰する。

6 関連資料との整合性

表 14: 関連資料との整合確認

資料	記載内容	本書の対応
docs/design/score_format.md	楽譜は各楽器マイコン内部で保持する.	ScoreEvent 配列を Arduino のプログラムメモリに埋め込む.
docs/design/protocol.md	指揮者から楽器は UDP, 楽器から PC は発音情報を送る.	CTRL / BEAT と NOTE を分け, NOTE は USB Serial 20 バイトで定義する.
ADR-0004	指揮 1 台, 楽器 4 台, 輪唱曲, 金管楽器を想定.	金管 3 声 + ドラム 1 声で「主旋律 3 + リズム 1」を満たす.
ADR-0007	指揮テンポ変更により演奏速度が追従する.	楽譜を拍単位で持ち, BPM から実時間へ変換する.
arduino_塩澤.pdf	ScoreEvent, startBeatNo, 20 バイト NOTE を定義.	同じ構造を採用し, 齋藤担当の最終仕様として整理する.
review_music_齋藤.md	休符, 輪唱入り, 通信形式, PROGMEM 方針が不足.	休符を noteNumber=0, 入り拍を startBeatNo, 通信を 20 バイト固定長に統一した.

7 今後の進め方

本書をもとに, 次の順で進める.

1. 第 3 回ミーティングで, 課題曲を「かえるのうた」とするか確定する.
2. 塩澤担当の ScoreEvent と NOTE 仕様を, 齋藤担当の正式仕様として採用することを合意する.
3. node_02~node_05 用の score_data.cpp を 4 ファイル作る.
4. Processing 担当へ, NOTE 20 バイト仕様と partId 対応表を渡す.
5. 1 パートだけで NoteOn / NoteOff が鳴ることを確認し, その後 4 パート輪唱へ広げる.

8 生成 AI の利用について

本書の作成には生成 AI を利用した. 主な利用目的は, 既存レビューの指摘整理, ルーブリック項目への対応漏れ確認, 塩澤担当仕様との整合確認, 文章構成の改善である. 最終的な仕様判断は, チーム内資料と WBS に基づいて行った.